

Encryption at rest across the storage services

Almost everything is encrypted by default now. The question the exam asks is who holds the key – and one default changed in April 2026.

Encrypted is table stakes. The real question is **who can make the data unreadable, and who can stop them.**

Four ways S3 encrypts on the server

OPTION	WHO HOLDS THE KEY	NOTES
SSE-S3	AWS, invisibly	The base level applied to every bucket. Free
SSE-KMS	You, via a KMS key	Auditable in CloudTrail, revocable, per-request KMS cost
DSSE-KMS	You, two independent layers	For requirements mandating dual-layer encryption
SSE-C	You supply it on every request	S3 never stores it. Disabled by default since April 2026

ON THE EXAM

The algorithm is AES-256 in all four. What differs is who holds the key, and therefore who controls access.

SSE-KMS, and the cost that surprises people

- ◆ Every object read or written calls KMS – **GenerateDataKey** on write, **Decrypt** on read. Each is a billable request.
- ◆ For a bucket with millions of small objects, that KMS bill can exceed the storage bill.
- ◆ **S3 Bucket Keys** fix it. S3 obtains a short-lived bucket-level key from KMS and uses it to derive data keys, cutting KMS requests by up to 99%.
- ◆ **Bucket Keys change the encryption context** from the object ARN to the bucket ARN, so CloudTrail shows fewer, coarser KMS events.
- ◆ Enable them at the bucket level. Existing objects keep whatever they were encrypted with.

ON THE EXAM

The trade is real: Bucket Keys cut cost and coarsen your audit trail. Know which one the scenario is asking about.

SSE-C changed on 6 April 2026

- ◆ **SSE-C is now disabled by default on all new S3 general purpose buckets**, and on existing buckets in accounts with no SSE-C encrypted data.
- ◆ The mechanism is **BlockedEncryptionTypes** in the bucket's default encryption configuration. New buckets have SSE-C blocked.
- ◆ To use it you must deliberately set **BlockedEncryptionTypes** to **NONE** via **PutBucketEncryption**.
- ◆ While blocked, any **PutObject**, **CopyObject**, **PostObject**, **multipart upload** or **replication** request specifying SSE-C is rejected with **403 AccessDenied**.
- ◆ Note replication. A cross-Region replication rule using SSE-C will start failing in a blocked bucket.

ON THE EXAM

Dated: any material written before April 2026 describes SSE-C as available by default. It is not, and that is deliberate.

Why AWS did that: SSE-C ransomware

- ◆ **S3 never stores an SSE-C key.** That is the whole design – and it is also the attack.
- ◆ An attacker with write access re-encrypts your objects under **their** key. AWS has no copy, so the data is unrecoverable without the attacker.
- ◆ Reported in the wild in early 2025. The April 2026 default is AWS's structural response to it.
- ◆ **Copy operations are S3 data events**, which module 1 told you are off by default – so the attack was often invisible.
- ◆ AWS's own guidance is now to keep SSE-C disabled unless a workload specifically requires it.

ON THE EXAM

The property that made SSE-C attractive – AWS never holds the key – is exactly what made it a ransomware primitive.

Enforcing encryption, and the client-side option

- ◆ Deny **s3:PutObject** when **s3:x-amz-server-side-encryption** is wrong or absent – absent needs **Null**, per module 20.
- ◆ Better: set the bucket's **default encryption**, so an unencrypted PUT is encrypted rather than rejected.
- ◆ **Client-side encryption** – the AWS Encryption SDK or S3 Encryption Client – means AWS never sees plaintext.
- ◆ It also means **no AWS service can read the data**. No Macie, no Athena, no Glue.
- ◆ Same for SSE-C: **AWS managed services cannot natively decrypt it**.

ON THE EXAM

Client-side is the strongest option and the most expensive in capability.

EBS, and the pattern that repeats

- ◆ **Encryption by default** is an account-and-Region setting. Every new volume and snapshot is then encrypted.
- ◆ **You cannot encrypt an existing volume in place.** Snapshot, copy with encryption, create a volume.
- ◆ Snapshots inherit encryption. Encrypted snapshots make encrypted volumes and AMIs.
- ◆ You can **re-encrypt under a different key** during a snapshot copy – the module 10 forensics workflow.
- ◆ **Encryption cannot be removed** once applied.

ON THE EXAM

Snapshot, copy encrypted, restore. The same three steps for EBS and RDS.

Encryption at rest, service by service

SERVICE	DEFAULT	CHANGE IT LATER?
S3	SSE-S3 on every bucket	Yes - change default encryption any time
EBS	Off unless you enable by default	No - snapshot, copy encrypted, restore
RDS and Aurora	Off. Set at creation	No - snapshot, copy encrypted, restore
DynamoDB	Always encrypted, AWS owned key	Yes - switch to AWS managed or customer managed
EFS	Set at creation	No - create a new file system and copy
FSx	Always encrypted at rest	Key choice at creation

ON THE EXAM

The third column is the exam column. 'We have an existing unencrypted database' is never answered by a setting change.

A media company's archive application uses SSE-C, because a broadcast regulator requires the company to hold the encryption keys itself in an on-premises key manager, and to be able to render archives unreadable by destroying them.

The team provisions a new bucket in May 2026 and every upload fails with HTTP 403 AccessDenied. The IAM role has s3:PutObject on the bucket, the bucket policy is permissive, and the same code works against a bucket created in 2024.

What is happening, and what should the team do?

- ☐ A SSE-C now requires a KMS key to be nominated; create one and reference it in the request
- ☐ B The role needs s3:PutObjectAcl in addition to s3:PutObject for SSE-C uploads
- ☐ C SSE-C is blocked by default on new buckets; set BlockedEncryptionTypes to NONE via PutBucketEncryption
- ☐ D SSE-C has been retired; migrate the archive to SSE-KMS with an imported key

◆ PAUSE AND DECIDE

What is happening, and what should the team do?

- A

 SSE-C now requires a KMS key
Contradictory. SSE-C exists precisely so the key is yours and never given to AWS; nominating a KMS key would make it SSE-KMS.
- B

 The role needs s3:PutObjectAcl
A permissions-shaped guess. The old bucket works with the identical role, so the difference is the bucket, not the policy.
- C

 SSE-C is blocked by default; unblock with PutBucketEncryption
Correct. Since 6 April 2026 new general purpose buckets block SSE-C. Workloads that genuinely need it must enable it deliberately.
- D

 SSE-C has been retired
Overreads the change. AWS disabled it by default and did not remove it – and the regulator's requirement rules out SSE-KMS anyway.

THE SIGNAL

'Works on a 2024 bucket, fails on a 2026 bucket, same code and same role' is a default that changed between them.

A logistics company discovers that an attacker with compromised credentials has copied every object in a data bucket over itself using SSE-C with the attacker's own key. The objects are now unreadable and AWS holds no copy of the key.

CloudTrail shows the credential being used, but no record of the copy operations themselves. Versioning was not enabled on the bucket.

What would have given the team visibility of the attack as it happened?

- ☐ A S3 data events on the bucket, which record CopyObject
- ☐ B GuardDuty S3 Protection, which would have flagged the encryption
- ☐ C AWS Config recording the bucket's encryption configuration changes
- ☐ D Macie automated sensitive data discovery on the bucket

◆ PAUSE AND DECIDE

What would have given the team visibility of the attack as it happened?

A S3 data events recording CopyObject

Correct. CopyObject is a data event, off by default, which is exactly why CloudTrail shows the credential but not the copies. This is the module 1 lesson in an incident.

B GuardDuty S3 Protection

S3 Protection consumes those same CloudTrail data events, so it is a plausible companion – but it detects patterns rather than giving the per-object record the team is missing.

C Config recording encryption configuration

The bucket's configuration never changed. SSE-C is specified per request by the caller, so nothing about the bucket looks different afterwards.

D Macie discovery

Macie answers what is in the data. It would report that it can no longer classify the objects, well after the fact.

THE SIGNAL

Option C is the instructive one: nothing about the bucket changed. SSE-C is a property of the request, not of the bucket.

An audit finds a production Aurora PostgreSQL cluster, created four years ago, that is not encrypted at rest. It holds patient records and must be encrypted with a customer managed KMS key. The application cannot tolerate more than a short maintenance window.

A junior engineer proposes enabling encryption on the existing cluster through the console.

What should the team do?

- A Enable encryption on the cluster and let Aurora encrypt the volumes in the background
- B Modify the cluster to add a KMS key, then reboot to apply it
- C Enable EBS encryption by default in the account, which will encrypt the cluster's storage
- D Snapshot the cluster, copy the snapshot specifying the KMS key, and restore from the encrypted copy

◆ PAUSE AND DECIDE

What should the team do?

- A

 Enable encryption in place
There is no such operation. RDS and Aurora encryption is set at creation and cannot be enabled on an existing cluster.
- B

 Modify and reboot to apply
The shape of a real RDS workflow – many parameter changes do apply on reboot. Encryption is not one of them.
- C

 EBS encryption by default
That setting governs EBS volumes you create. It has no effect on existing storage, and Aurora's storage is not an EBS volume you manage.
- D

 Snapshot, copy specifying the key, restore
Correct. The standard three-step pattern – and the copy step is where you nominate the customer managed key.

THE SIGNAL

Snapshot, copy encrypted, restore. The same three steps as EBS, and the answer whenever encryption was not set at creation.

A telemetry platform writes roughly 400 million small objects a month into an S3 bucket encrypted with SSE-KMS using a customer managed key. Finance flags that the KMS charges now exceed the S3 storage charges.

Security requires that the data stays encrypted under a customer managed key that the company can disable, and that key usage remains auditable.

What should the platform implement?

- ☐ A Switch the bucket to SSE-S3, which removes the KMS charges entirely
- ☐ B Enable S3 Bucket Keys on the bucket
- ☐ C Batch the objects into larger archives before upload
- ☐ D Move to SSE-C, since customer-provided keys incur no KMS charges

◆ PAUSE AND DECIDE

What should the platform implement?

A Switch to SSE-S3

Removes the charge and the control. Security requires a key the company can disable, and SSE-S3 keys are AWS's - there is nothing to disable.

C Batch into larger archives

Genuinely effective and an application redesign, with real consequences for how the telemetry is queried. Bucket Keys need one setting.

B Enable S3 Bucket Keys

Correct. Up to 99% fewer KMS requests, while the data stays under the same customer managed key. The audit trail becomes coarser, not absent.

D Move to SSE-C

Removes KMS charges and every auditable record of key use, requires the key on every request, and is blocked by default since April 2026.

THE SIGNAL

Two requirements survive the cost problem: a key they can disable, and auditability. Only Bucket Keys keeps both.

Six things to carry forward

- ◆ **The algorithm is the same in all four S3 options.** What differs is who holds the key.
- ◆ **SSE-C disabled by default since 6 April 2026.** Unblock deliberately with PutBucketEncryption.
- ◆ **SSE-C ransomware is why.** AWS holds no copy, and CopyObject is a data event.
- ◆ **Snapshot, copy encrypted, restore** – for EBS and RDS alike.
- ◆ **Bucket Keys cut KMS cost up to 99%** and coarsen the audit trail to the bucket ARN.
- ◆ **Client-side and SSE-C both stop AWS services reading the data.**

ON THE EXAM

Next: integrity and retention – Object Lock, Vault Lock, Backup and ransomware recovery.